

HelixSpecifier

HelixSpecifier

Tagline: Spec-driven development that scales its own ceremony to the work.

Summary: HelixSpecifier is a Go engine that fuses three development methodologies — SpecKit’s spec-driven workflow, Superpowers’ TDD discipline, and GSD’s milestone lifecycle — into one adaptive flow. It classifies each task by effort and scales the amount of process up or down accordingly.

Short description (~40 words): HelixSpecifier is a spec-driven development fusion engine for AI agents. It combines SpecKit, Superpowers and GSD, classifies work by effort level, runs debate-backed specification phases, enforces a minimum test-to-code ratio, and learns from every completed flow.

Long description (150-250 words): HelixSpecifier is a spec-driven development (SDD) fusion engine written in Go (module `digital.vasic.helixspecifier`), built as a component of the HelixAgent AI ensemble. It unifies three complementary development pillars into a single adaptive workflow: SpecKit’s seven-phase SDD process (Constitution, Specify, Clarify, Plan, Tasks, Analyze, Implement), Superpowers’ test-driven discipline with parallel subagent execution, and GSD’s milestone lifecycle management.

Its central idea is *adaptive ceremony*: the engine classifies incoming work by effort level and scales the amount of process to match, so a small change does not drag the full ceremony of a large feature. Ten power features build on this — bounded-concurrency parallel task execution, a machine-readable “Constitution as Code” that enforces mandatory rules, “Nyquist TDD” that tracks and enforces a minimum test-to-implementation ratio, multi-round multi-agent debate for specification refinement, adaptive skill-proficiency learning, brownfield analysis of legacy code, predictive specification from historical patterns, cross-project knowledge transfer, runtime ceremony adjustment, and a persistent spec memory with semantic search.

It is consumed as a Go module, either via `go get` or a local `replace` directive, and exposes a simple engine API: register the three pillars plus a ceremony scaler and spec memory, classify effort, then execute the full flow and receive a quality-scored result. Like the rest of the Helix family, it is developed under an anti-bluff verification regime with an in-process challenge runner over real code.

Why we built it: Spec-driven development, rigorous TDD, and milestone management are usually three separate practices with three separate tools. HelixSpecifier was built so an AI agent (HelixAgent) can run all three as one coherent, self-scaling workflow instead of stitching them together by hand.

Why it's a game-changer: It makes process proportional to the work. Instead of applying heavyweight ceremony everywhere (slow) or nowhere (risky), the engine sizes the ceremony to the effort automatically — and backs specification decisions with a multi-agent debate rather than a single opinion.

What's innovative: - **Adaptive ceremony** driven by real-time quality metrics. - **Nyquist TDD** — a test-to-implementation ratio gate (minimum 2x) inspired by the Nyquist sampling theorem. - **Debate architecture** — multi-round, multi-agent specification refinement with position scoring. - **Predictive specification** and **cross-project transfer** that learn from accumulated flows. - **Constitution as Code** — machine-readable, enforced project rules.

Biggest technical challenges + how solved: - *Merging three methodologies without conflict* — solved with a fusion engine that registers each pillar behind a common interface and orchestrates them through a single flow lifecycle. - *Deciding how much process a task needs* — solved with an effort classifier plus a ceremony scaler that adjusts dynamically during execution. - *Keeping specification quality high* — solved with debate-backed refinement (position scoring across rounds) and enforced TDD ratios.

Tech stack: - **Go** — single-binary/importable engine; concurrency powers bounded-parallel task dispatch and the debate rounds. - **logrus** — structured logging across the engine and pillars. - **SpecKit pillar** — seven-phase spec-driven-development process implementation. - **Superpowers pillar** — TDD discipline with parallel subagent execution. - **GSD pillar** — milestone/lifecycle management. - **Spec memory store** — persistent, semantically searchable specification index for learning and transfer.

Public links: - GitHub: <https://github.com/HelixDevelopment/specifier> (public; the display name “HelixSpecifier” maps to the repo specifier) - License: no LICENSE detected via the GitHub API — UNVERIFIED / not declared.

Suggested diagrams/illustrations (OpenDesign): 1. Three-pillars-to-one-flow diagram (SpecKit + Superpowers + GSD → fusion engine → executed flow with quality score). 2. Adaptive-ceremony dial: effort level in → ceremony level out, shown as a scaling gauge. 3. Debate architecture: multiple agents proposing/scoring positions across rounds converging on a spec.

Site relevance: both (vasic.digital as an AI developer-tooling product; milosvasic.ru as an AI-engineering / methodology highlight).

Priority tier: Helix-primary.

Source provenance: - gh api repos/HelixDevelopment/specifier (metadata: public, Go, no license, created 2026-02-24, pushed 2026-06-24). - gh api repos/HelixDevelopment/specifier/readme (three pillars, 10 features, API usage, effort classification). - /Volumes/T7/Projects/vasic/_analysis/github-helix-others.{md,json} (description, HelixAgent relationship). - UNVERIFIED: license not declared. "Debate ensemble" positioning taken from README + harvested notes.